



20. Advanced Page Tables

외우자

Paging: Linear Tables

Problem

Hybrid Approach: Paging & Segments

TLB miss on Hybrid Approach

Problem of Hybrid Approach

Multi-level Page Tables

Page Directory (PD)

Advantage

Disadvantage

Example

Example: Single level paging

Example: two level paging

Page directory index

정리

Page Directory Index (PDIIndex)

Page Table Index (PTIndex)

Example: More than Two Levels

Page Table Index

Single level page table & two level page table

More than Two Level: Page Directory

three level page table

Multi-level Page Table Control Flow

Exercise

Assumption

Inverted Page Tables

Naïve approach

Better: Find possible matches entries by hashing vpn+asid

Summary

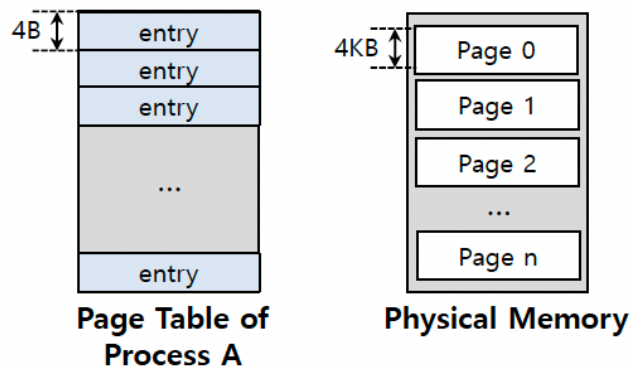
외우자

- $2^{10} = 1024 = K$
 - $2^{15} = 2^5 * K = 32K$

- $2^{20} = M$
- $2^{30} = G$
 - $2^{33} = 2^3 * G = 8G$
- $2^{40} = T$

Paging: Linear Tables

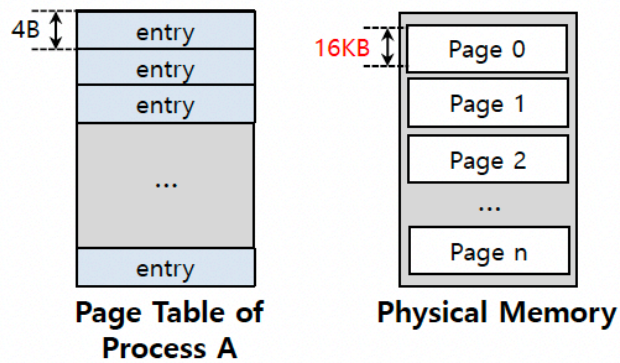
- We usually have one page table for every process in the system.
 - Assume that **32-bit address space(4GB의 Virtual Address Space겠지?)** with **4KB(2^{12}) pages** and **4byte page-table entry**.



- Page table size = $2^{32} / 2^{12} * 4\text{byte} = 2^{20} * 4\text{byte} = 4\text{MByte}$

| Page table are too big and thus consume too much memory.

- Assume that **32-bit address space** with **16KB pages** and **4-byte page-table entry**.



$$\frac{2^{32}}{2^{14}} * 4 = 1MB \text{ per page table}$$

- $2^{32} / (2^4 * 2^{10}) * 4\text{byte} = 2^{18} * 2^2 \text{ byte} = 2^{20} \text{ byte} = 2\text{MByte?}$

그림 숫자 잘못됨?

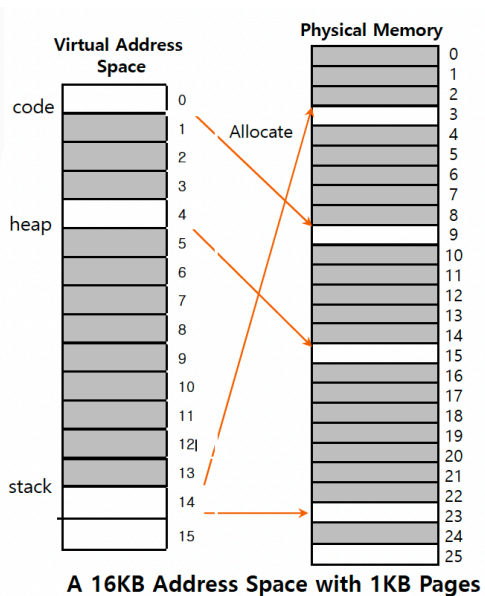
Big pages lead to internal fragmentation.

Big Page를 통해 **Page Table** 크기 자체는 줄었지만, **Internal Fragmentation**이 심해 짐

Problem

- Single page table for the address space of process.
- Most of the page table is **unused**, full of invalid entries.

Page Table의 대부분은 Unused $\Rightarrow$ Invalid Entries

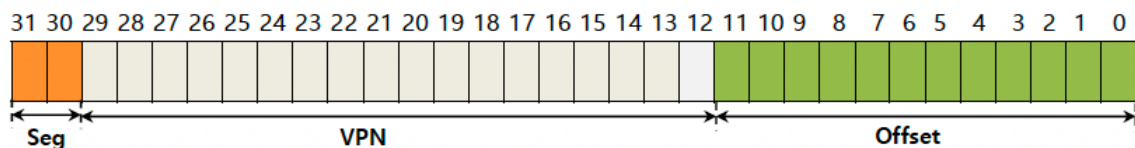


PFN	valid	prot	present	dirty
10	1	r-x	1	0
-	0	-	-	-
-	0	-	-	-
-	0	-	-	-
15	1	rw-	1	1
...	...	...	...	...
-	0	-	-	-
3	1	rw-	1	1
23	1	rw-	1	1

A Page Table For 16KB Address Space

Hybrid Approach: Paging & Segments

- **Page table for each segment**
 - segment table에는 각 segment의 page table에 대한 base와 bound register
 - **The base register**: physical address of a linear page table for that segment.
 - **The bound register**: end(size) of the page table.
- Example: Each process has three page tables associated with it.
 - 32-bit Virtual Address Space with 4KB pages ⇒ offset bits는 12개



Segment Value	Content
00	unused segment
01	code
10	heap
11	stack

- Segment를 총 3개를 두었기 때문에 Segment Table Index를 위한 Bits를 2개 마련해두었음
 - Page의 개수가 당연히 조금 줄어들 것
 - 대신, Page Table의 Size도 줄어들 것

TLB miss on Hybrid Approach

- The hardware gets physical address from page table.
 - The hardware uses the **segment bits(SN)** to determine which base and bounds pair to use.
 - The hardware then takes the physical address therein and combines it with the VPN as follows to form the address of the page table entry(PTE)

(Code=00, Heap=01, Stack=11) ⇒ 가정

$SN = (VirtualAddress \& SEG_MASK) \gg SN_SHIFT$

예시)

VA = 0x01234567 (32비트)

SEG_MASK = 0xC0000000 (1100 0000 0000 0000 0000 0000 0000 0000)

SN_SHIFT = 30

$SN = (0x01234567 \& 0xC0000000) \gg 30$
= 0 (Segment 0번)

$VPN = (VirtualAddress \& VPN_MASK) \gg VPN_SHIFT$

VPN_MASK = 0x3FFFF000 (0011 1111 1111 1111 0000 0000 0000 0000)

VPN_SHIFT = 12

$VPN = (0x01234567 \& 0x3FFFF000) \gg 12$
= 0x01234000 >> 12
= 0x01234 (code segment 내에서 0x01234번째 페이지)

$AddressOfPTE = Base[SN] + (VPN * sizeof(PTE))$

Base[0] = 0x10000000 // Code 세그먼트 페이지 테이블 주소

Base[1] = 0x20000000 // Heap 세그먼트 페이지 테이블 주소

Base[3] = 0x30000000 // Stack 세그먼트 페이지 테이블 주소

sizeof(PTE) = 4 bytes

$AddressOfPTE = Base[0] + (0x1234 * 4)$
= 0x10000000 + 0x48D0
= 0x100048D0

Problem of Hybrid Approach

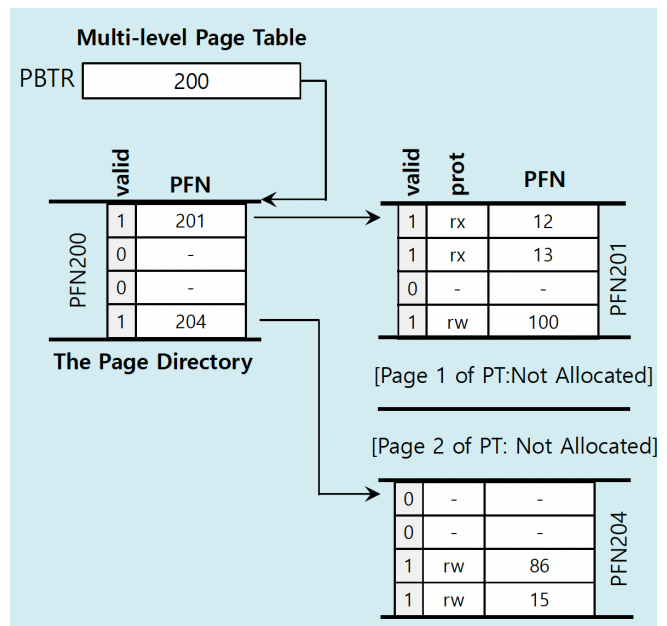
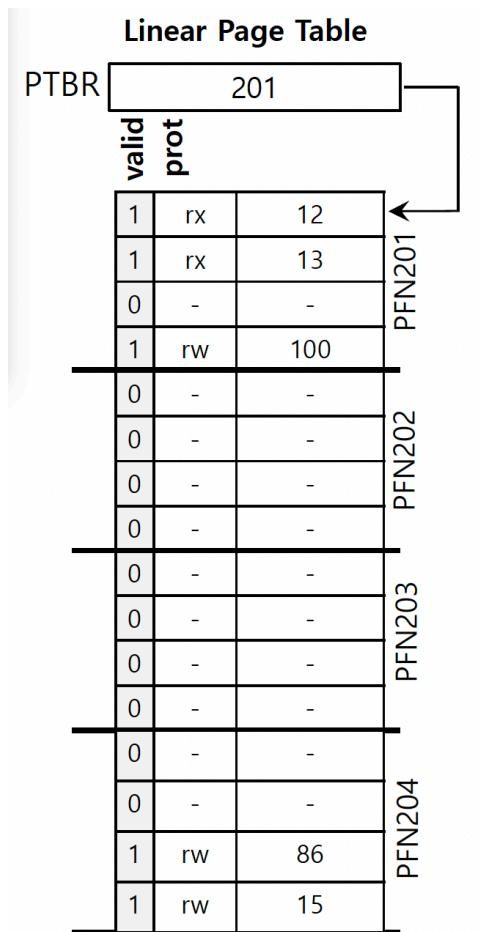
- Hybrid Approach is not without problems.
 - If we have a large but sparsely-used heap, we can still end up with a lot of page table waste.
 - **Causing external fragmentation to arise again.**
Heap PT가 엄청 크지만 sparse하게 사용 - 이걸 internal fragmentation, 페이지

지 테이블들 사이에 못 쓰는 공간 생김, 세그먼트(페이지 테이블)가 **할당 단위**니까 **외부 단편화**

- 인터넷에서 찾은거: Code, Stack Segment의 낭비는 줄일 수 있지만, **Heap Segment의 낭비는 막을 수 없다.**
malloc/free로 중간중간 구멍 생김 (단편화), 사용 패턴이 불규칙함 등등

Multi-level Page Tables

- Turn the linear page table into something like a tree.
 - Chop up the **page table into page-sized units.**
Page Table을 Page-Size의 특정 Unit(Page)으로 쪼개자
 - If an entire page of page-table entries is invalid, don't allocate that page of the page table at all
특정 Page의 PTE들이 모두 Invalid이면, 그 Page는 애초에 할당하지 말자
 - To track whether a page of the page table is valid, use a new structure, called **page directory.**
PDE : Valid Bit + PFN



Page Directory (PD)

- 일종의 "목차" 역할
- Page Table들의 목록을 관리해
- 여러 PDE들로 구성
- 각 엔트리(PDE)는 하나의 page table이 어디 있는지 가리켜
 - **Valid bit:** 이 page table이 실제로 존재하는지 표시
 - **PFN (Page Frame Number):** 그 page table이 물리 메모리 어디에 있는지
- PDE엔 Page Table을 기록하고 있는 Physical Frame의 PFN이 있고, 그 PFN을 따라가서 찾은 Frame의 PTE는 VPNtoPFN Translation 정보가 들어있다

Advantage

- Only allocates page-table space in proportion to the amount of address space you are using
사용하는 address space 만큼 page-table 공간을 할당한다!
- The OS can grab the **next free page** when it needs to allocate or grow a page table.
확장할 때 용이

Disadvantage

- Multi-level table is a small example of a **time-space trade-off**.
기존 Linear Page Table 방식에 비해 공간 효율은 늘어나지만 Access 횟수가 늘어나기 때문에 시간 효율은 떨어진다
- **Complexity**.

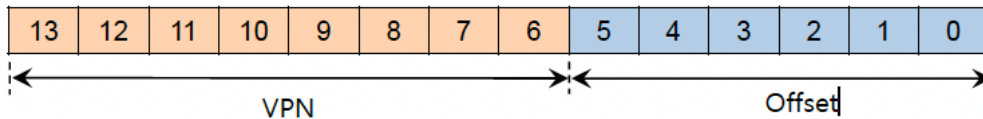
Example

0000 0000	code
0000 0001	code
...	(free)
	(free)
0000 0100	heap
0000 0101	heap
.....	(free)
	(free)
1111 1110	stack
1111 1111	stack

- Page 0, 1: code
- Page 4, 5: heap
- Page 254, 255: stack

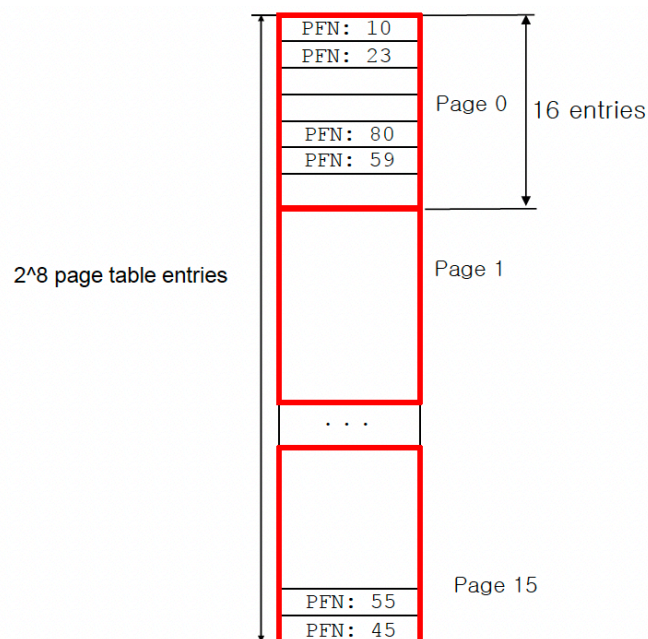
A 16KB Address Space With 64-byte Pages

Flag	Detail
Address Space	16 KB (2^{14} Byte)
Page Size	64 byte (2^6 Byte)
Virtual Address	14 bit
VPN	8 bit
Offset	6 bit
Page table entry	4 byte



Example: Single level paging

- 256 page table entries: 2^8 entries
- Page table size: $256 * 4 \text{ Byte} = 1 \text{ Kbyte}$
- Sixteen pages (64 byte each): $1024/64 = 16$
 왜 64로 나누냐? → page크기로 나누는거잖아~
 따라서 Page Table은 16개의 page로 표현할 수 있다
 그말인즉슨 PDE(Page Directory Entry)가 16개다!



Example: two level paging

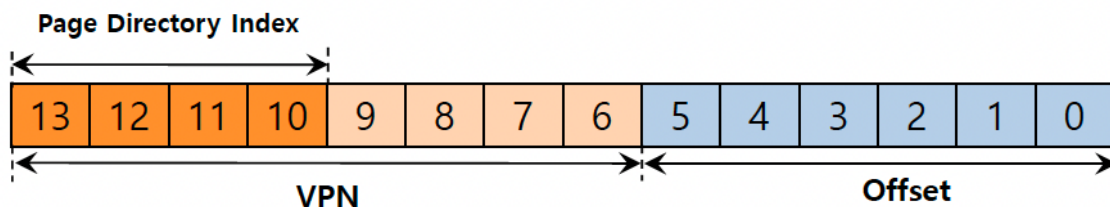
Page directory index

- It is used to find the address of the page table entry
- A page table consists of 16 pages.
- 16 entries for page directory: one entry per page of the page table.
- $16 * 4 \text{ byte} = 64 \text{ byte}$ is required for page directory. → it can fit in a page.
 PDE가 16개잖아

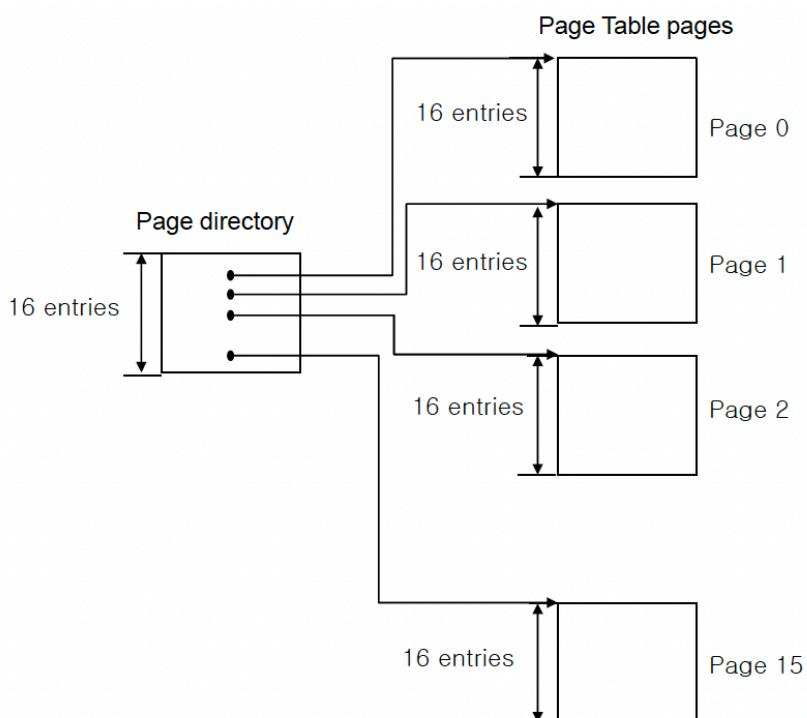
그래서 bit가 4개 필요한건가

- 4 bits for page directory index.

★ $PTEAddr = (PDE.PFN \ll SHIFT) + (PTIndex * sizeof(PTE))$

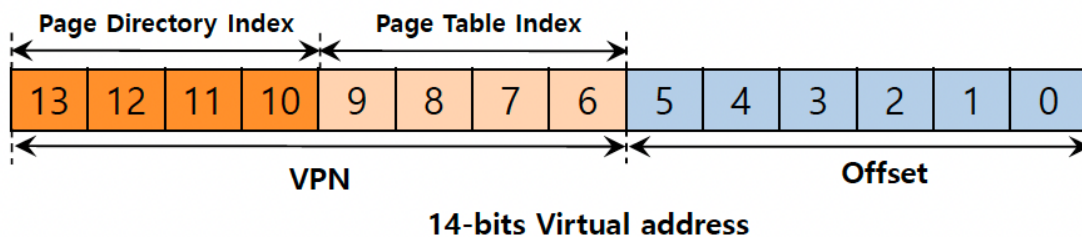
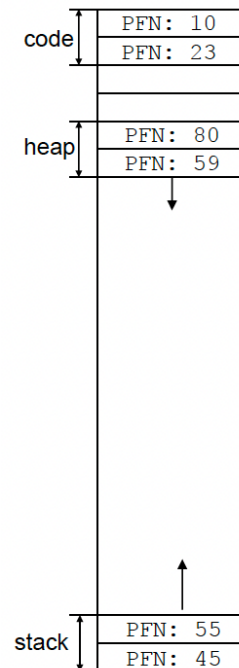


- If the page-directory entry is invalid, raise an exception (The access is invalid).



- Single level paging: 16 pages ↔ Two level paging: 3 pages

Page Directory		Page of PT (@PFN:100)			Page of PT (@PFN:101)		
PFN	valid?	PFN	valid	prot	PFN	valid	prot
100	1	10	1	r-x	—	0	—
—	0	23	1	r-x	—	0	—
—	0	—	0	—	—	0	—
—	0	—	0	—	—	0	—
—	0	80	1	rw-	—	0	—
—	0	59	1	rw-	—	0	—
—	0	—	0	—	—	0	—
—	0	—	0	—	—	0	—
—	0	—	0	—	—	0	—
—	0	—	0	—	—	0	—
—	0	—	0	—	—	0	—
—	0	—	0	—	—	0	—
—	0	—	0	—	—	0	—
—	0	—	0	—	—	0	—
—	0	—	0	—	55	1	rw-
101	1	—	0	—	45	1	rw-



정리

Page Directory Index (PDIndex)

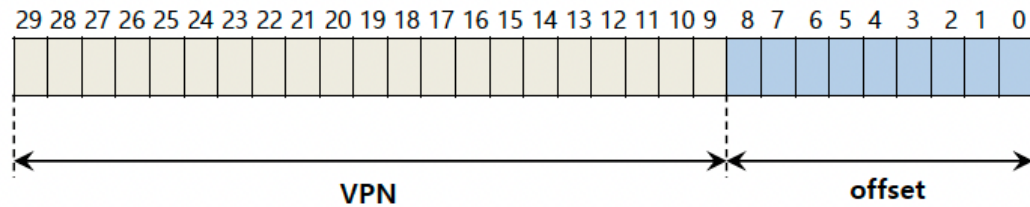
- VPN에서 PDE의 Address를 찾는데 사용되는 부분이다.
- $PDEAddr = PageDirBase + (PDIndex \times sizeof(PDE))$

Page Table Index (PTIndex)

- PDE가 가리키는 Valid Page of Page Table에서 PTE의 Address를 찾는데 사용된다.
- $PTEAddr = (PDE.PFN \ll SHIFT) + (PTIndex \times sizeof(PTE))$ ★★★
- $PhysAddr = (PTE.PFN \ll SHIFT) + Offset$ ★★★

Example: More than Two Levels

- In some cases, a deeper tree is required.
- Consider the following address space.

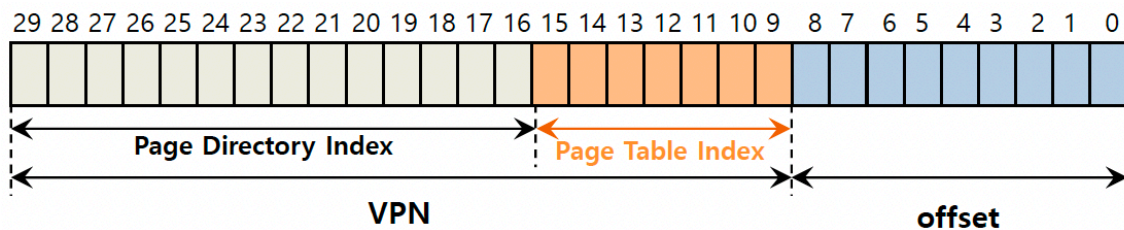


Flag	Detail
Virtual Address	30 bit
Page Size	512 byte (2^9)
VPN	21 bit
Offset	9 bit
PTE 크기 가정	4 byte (2^2)

- page table 크기 $\Rightarrow 2^2(\text{PTE 크기}) * 2^{21}(\text{PTE 개수}) = 2^{23}\text{Byte} = 2^3\text{MB}$
- 그렇다면?(Two-Level)
 - 몇 개로 나눌 수 있냐 $2^{23} / 2^9 = 2^{14}$ 개의 pages
 - 한 page당 2^7 개의 PTE가 들어가는거

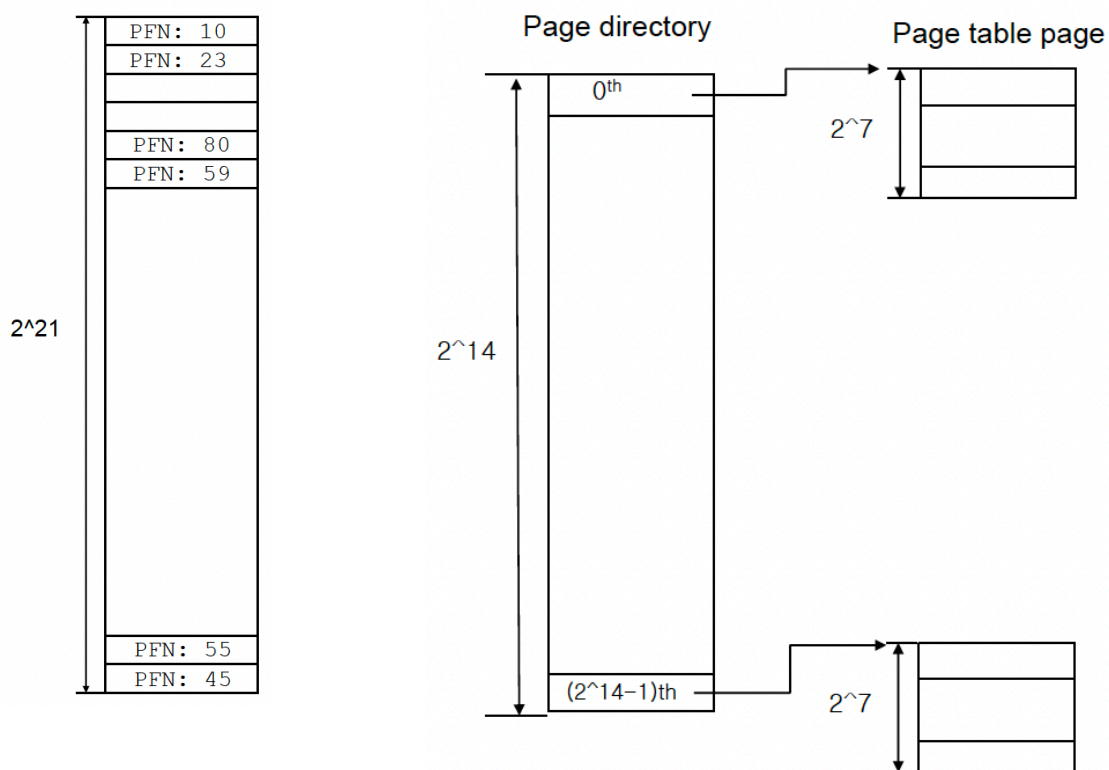
Page Table Index

- Number of pages in a page table
 - 2^{21} page table entries
 - 2^7 (128) PTE's in a single page (512/4, 512 page size, 4 byte pte size)
 - 2^{14} pages in a page table
- Number of page directory entries: 2^{14}



Flag	Detail
Virtual Address	30 bit
Page Size	512 byte
VPN	21 bit
Offset	9 bit
Page entry per page	128 PTEs ($512 / 4 = 2^7$)

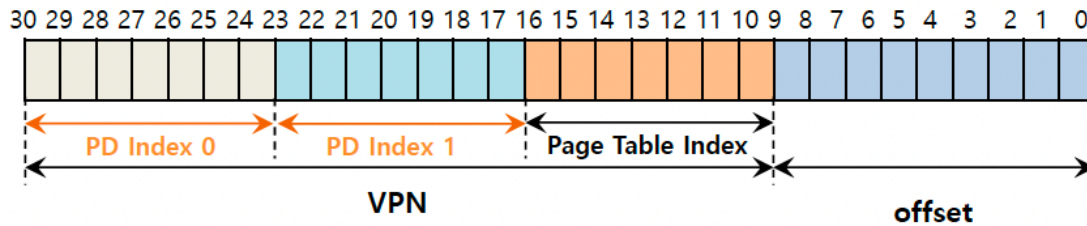
Single level page table & two level page table



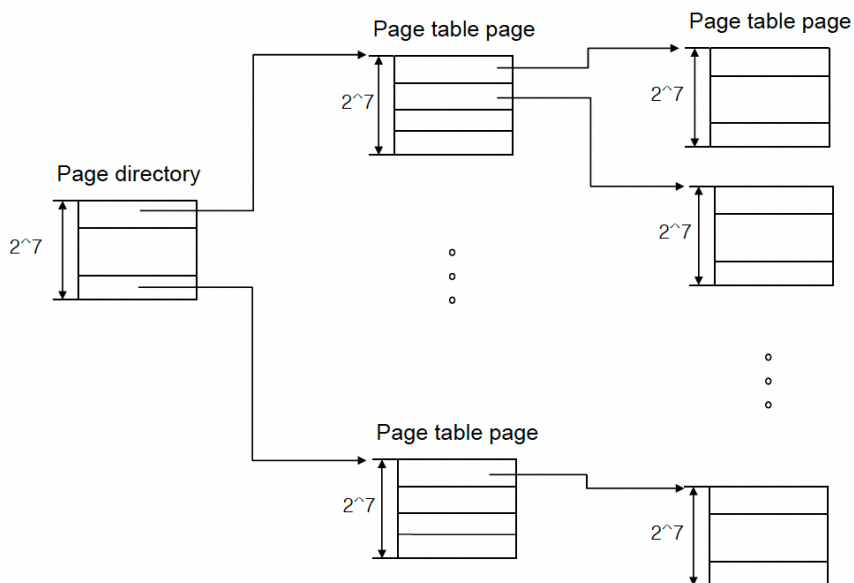
More than Two Level: Page Directory

- The number of pages in a page directory: $2^{14}/2^7 = 2^7$ (128)

- Page the page directory



three level page table



Multi-level Page Table Control Flow

```

VPN = (VirtualAddress & VPN_MASK) >> SHIFT
(Success, TlbEntry) = TLB_Lookup(VPN)
if(Success == True) // TLB Hit
    if(CanAccess(TlbEntry.ProtectBits) == True)
        Offset = VirtualAddress & OFFSET_MASK
        PhysAddr = (TlbEntry.PFN << SHIFT) | Offset // 그냥 TLB entry 그대로
        사용
        Register = AccessMemory(PhysAddr)
    else RaiseException(PROTECTION_FAULT);
/* TLB Miss */
else {
    PDIndex = (VPN & PD_MASK) >> PD_SHIFT // Page Directory Index

```

```

PDEAddr = PDBR + (PDIndex * sizeof(PDE))    // PDE의 Address 계산
PDE = AccessMemory(PDEAddr)                  // PDE 추출
if(PDE.Valid == False)                        // Valid Page가 없는 곳을
    RaiseException(SEGMENTATION_FAULT)        // 접근하려고 하면 segfau
lt
else { // PDE is Valid: now fetch PTE from PT
    PTIndex = (VPN & PT_MASK) >> PT_SHIFT
    PTEAddr = (PDE.PFN << SHIFT) + (PTIndex * sizeof(PTE))
    PTE = AccessMemory(PTEAddr)

    if(PTE.Valid == False)
        RaiseException(SEGMENTATION_FAULT)
    else if(CanAccess(PTE.ProtectBits) == False)
        RaiseException(PROTECTION_FAULT);
    else {
        TLB_Insert(VPN, PTE.PFN , PTE.ProtectBits)
        RetryInstruction()
    }
}
}

```

- **(1 lines)** extract the virtual page number(VPN)
- **(2 lines)** check if the TLB holds the translation for this VPN
- **(5-8 lines)** extract the page frame number from the relevant TLB entry, and form the desired physical address and access memory
- **(11 lines)** extract the Page Directory Index(PDIndex)
- **(13 lines)** get Page Directory Entry(PDE)
- **(15-17 lines)** Check PDE valid flag. If valid flag is true, fetch Page Table entry from Page Table

Exercise

Assumption

- Logical address : 64 bit

- Page size : 32KB = $2^5 * 2^{10} = 2^{15}$ byte
- PTE : 4B = 4byte

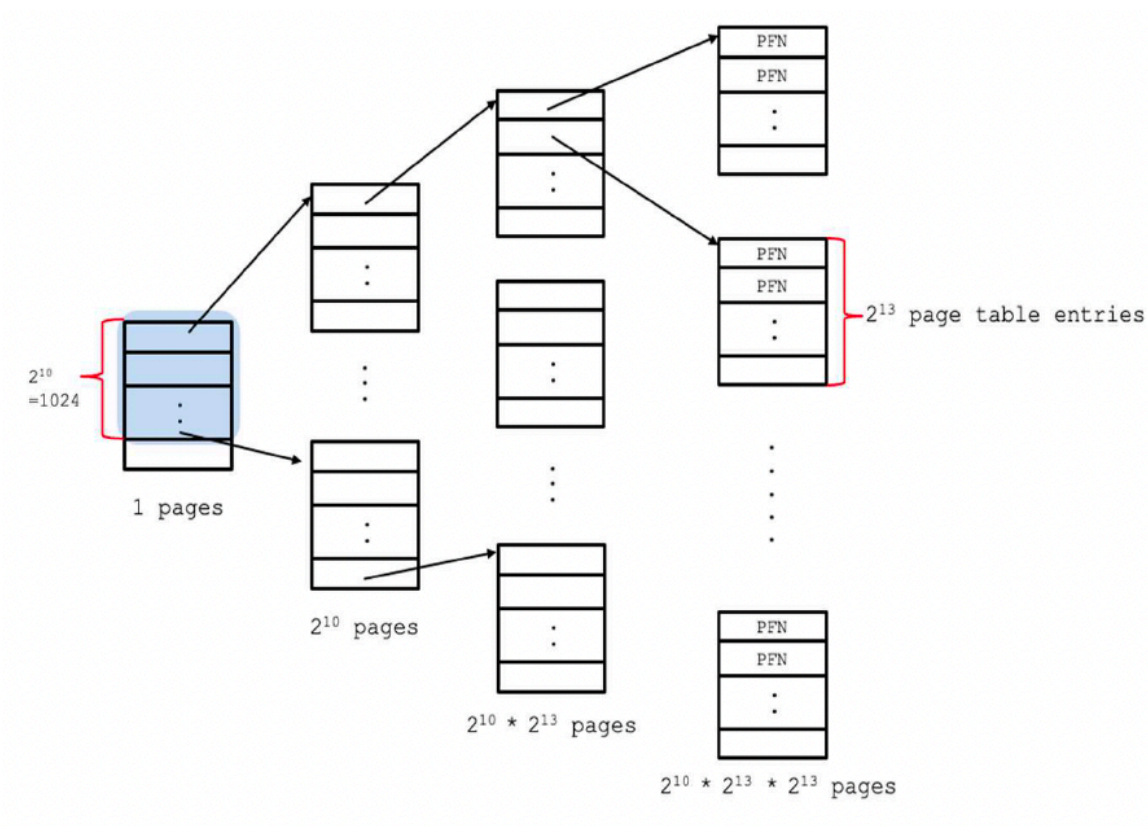
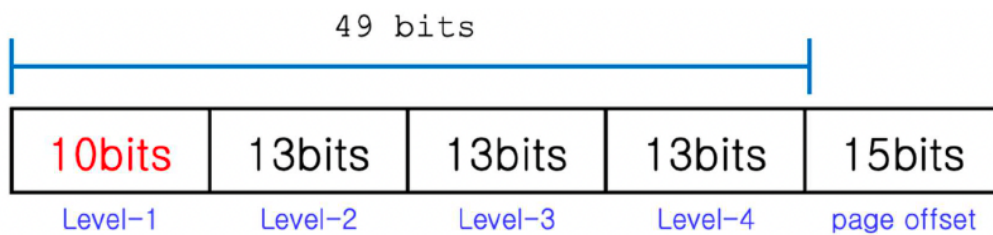
1. How many bits for page offset?: 15bits

$32\text{ K} = 2^{15} \rightarrow$ page offset: 15 bits $\rightarrow$ **VPN : 49 bits**

2. How many page table entries in each page?

page size / PTE = $32\text{ KB} / 4\text{ B} = 2^{15} / 2^2 = 2^{13} \rightarrow$ 13 bits 이계 Page Table Index?

2^{13} 개



1. Total number of pages for this page table?

a. page dir (level-1) $\Rightarrow 1$

b. page table(level-2) $\Rightarrow 2^{10}$

c. page table(level-3) $\Rightarrow 2^{10} * 2^{13}$

d. page table(level-4) $\Rightarrow 2^{10} * 2^{13} * 2^{13}$

$$1 + 2^{10} + 2^{10} * 2^{13} + 2^{10} * 2^{13} * 2^{13}$$

2. How many frames can this page table address?

: the maximum number of **physical frames this structure can reference**

이 Page Table의 총 Entry 개수를 보면 되는데, 이 System의 Logical Address에서 VPN이 총 몇 Bits인지 보면 된다

$$2^{10} * 2^{13} * 2^{13} * 2^{13} = 2^{49}$$

3. Page table size?

: total number of pages * page size

$$(1 + 2^{10} + 2^{10} * 2^{13} + 2^{10} * 2^{13} * 2^{13}) * 2^{13}$$

4. How many memory accesses to access a certain page?

: if TLB miss, need to read each level and read physical address

5 times

Inverted Page Tables

- 실제 물리 메모리 프레임 개수만큼만 엔트리를 만드는 거야. 즉, 가상 페이지 수가 아니라 물리 프레임 수에 비례하게 만드는 거지.
- 구조
 - 각 엔트리는 (프로세스 ID, 가상 페이지 번호) 쌍을 저장
 - 물리 프레임 번호가 인덱스가 되고, 그 프레임에 어떤 프로세스의 어떤 가상 페이지가 들어있는지를 기록
- Keeping a single page table that has an entry for each physical page of the system.
- The entry tells us which process is using this page, and which virtual page of that process maps to this physical page.
엔트리는 "어떤 프로세스가 이 페이지를 쓰고 있는지", 그리고 "그 프로세스의 어떤 가상 페이지가 이 물리 페이지에 매핑되는지"를 알려줘.
- Only need entries for virtual pages w/ valid physical mappings
유효한 물리 매핑이 있는 가상 페이지들에 대해서만 엔트리가 필요해

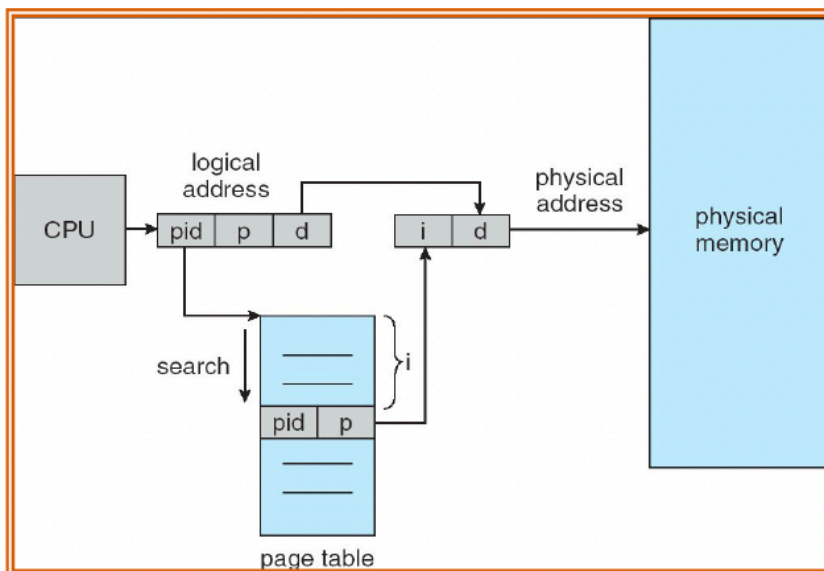
- One entry for each real page of memory
메모리의 실제 페이지마다 하나의 엔트리만 있으면 돼

Naïve approach

- Search through data structure $\langle \text{ppn}, \text{vpn} + \text{asid} \rangle$ to find match
- Too much time to search entire table

Better: Find possible matches entries by hashing $\text{vpn} + \text{asid}$

- Smaller number of entries to search for exact match



Summary

- Reducing the page table size
- Per segment page table
- Multi-level paging
 - Efficient use of memory
 - Severe TLB miss